

Discussion of Exercise Sheet 2

Exercise 2.1 Induction for inequalities.

(a) Prove via mathematical induction that for all integers $n \geq 5$,

$$2^n > n^2.$$

In your solution, you should address the base case, the induction hypothesis and the induction step.

Base Case: Sei $k=5$

$$2^5 = 32 > 25 = 5^2 \text{ stimmt, deshalb gilt die Aussage f\"ur } k=5$$

Induktions-Hypothese: Wir nehmen an, dass die Aussage f\"ur ein beliebiges $k \in \mathbb{N}$, $k \geq 5$ gilt.

$$\text{d.h. } 2^k > k^2$$

Induktions-Schritt: Wir wollen die Aussage f\"ur $k+1$ zeigen, d.h. $2^{k+1} > (k+1)^2$

$$2^{k+1} = 2 \cdot 2^k$$

$$\stackrel{\text{IH}}{>} 2 \cdot k^2$$

$$= k^2 + k^2$$

$$\hookrightarrow k^2 = k \cdot k \geq 5k \text{ weil } k \geq 5$$

$$\geq k^2 + 5k$$

$$= k^2 + 2k + 3k$$

$$\hookrightarrow 3k \geq 15 > 1 \text{ weil } k \geq 5$$

$$> k^2 + 2k + 1$$

$$\hookrightarrow \text{binomische Formel}$$

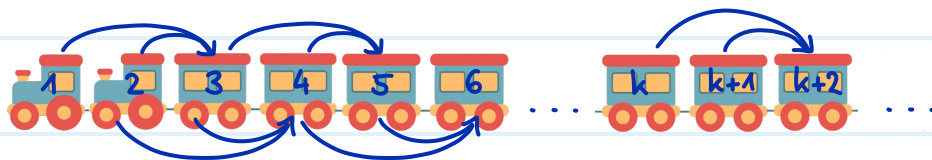
$$= (k+1)^2$$

Somit ist die Aussage f\"ur alle $n \geq 5$ bewiesen per Induktion.

(b) Recall that the Fibonacci numbers are defined by $f_0 = 0$, $f_1 = 1$ and the recursion relation $f_{n+1} = f_n + f_{n-1}$ for all $n \geq 1$. For example, $f_2 = 1$, $f_5 = 5$, $f_{10} = 55$, $f_{15} = 610$.

Prove via mathematical induction that $f_n \geq \frac{1}{3} \cdot 1.5^n$ for $n \geq 1$.

In your solution, you should address the base case, the induction hypothesis and the induction step.



Base Case: Sei $k=1$. $f_1 = 1 \geq 0.5 = \frac{1}{3} \cdot 1.5^1$ stimmt, deshalb gilt die Aussage für $k=1$

Sei $k=2$. $f_2 = 1 \geq 0.75 = \frac{1}{3} \cdot 1.5^2$ stimmt, deshalb gilt die Aussage für $k=2$

Induktions-Hypothese: Wir nehmen an, dass die Aussage für ein beliebiges $k \in \mathbb{N}$ und $k+1$ gilt,

$$\text{d.h. } f_k \geq \frac{1}{3} \cdot 1.5^k \text{ und } f_{k+1} \geq \frac{1}{3} \cdot 1.5^{k+1}$$

Induktions-Schritt: Wir wollen die Aussage für $k+2$ zeigen, d.h. $f_{k+2} \geq \frac{1}{3} \cdot 1.5^{k+2}$

$$\begin{aligned} f_{k+2} &\stackrel{\text{def}}{=} f_{k+1} + f_k \\ &\stackrel{\text{IH}}{\geq} \frac{1}{3} \cdot 1.5^{k+1} + \frac{1}{3} \cdot 1.5^k && \text{ausklammern} \\ &= \frac{1}{3} \cdot 1.5^k (1.5 + 1) \\ &= \frac{1}{3} \cdot 1.5^k \cdot 2.5 && 1.5^2 = 2.25 < 2.5 \\ &> \frac{1}{3} \cdot 1.5^k \cdot 1.5^2 && \text{Potenzgesetz} \\ &= \frac{1}{3} \cdot 1.5^{k+2} \end{aligned}$$

Somit ist die Aussage für alle $n \geq 1$ bewiesen per Induktion.

Exercise 2.2 Induction for algorithms (2 bonus points).

We are given a list $a_1, \dots, a_n \in \mathbb{N}$ of $n \geq 1$ distinct integers and our task is to find the maximum element a^* in this list:

- (i) $a^* \in \{a_1, \dots, a_n\}$.
- (ii) $a^* \geq a_i$ for all $i = 1, \dots, n$.

Consider the following recursive algorithm:

Algorithm 1 Recursive maximum

```

if  $n = 1$  then
  return  $a_1$ 
else
  Apply this algorithm recursively and find the maximum element  $a'$  among  $a_1, \dots, a_{n-1}$ 
  if  $a' > a_n$  then
    return  $a'$ 
  else
    return  $a_n$ 

```

$\triangleright n \geq 2$
 $\triangleright a' \leq a_n$

Prove via mathematical induction that this algorithm correctly returns the maximum element of the given list of numbers.

Aussage: Der Algorithmus gibt das Maximum der Liste korrekt zurück

Base Case: Sei $k=1$ Der Algorithmus gibt a_1 zurück, welches das einzige Element der Liste ist. Es gilt $a_1 \in \{a_1\}$ und $a_1 \geq a_1$, d.h. (1) und (2) sind erfüllt. Deshalb gilt die Aussage für $k=1$.

Induktions-Hypothese: Wir nehmen an, dass die Aussage für ein beliebiges $k \in \mathbb{N}$, $k \geq 1$ gilt.
d.h. "Der Algorithmus gibt das Maximum der Liste korrekt zurück für Länge k "

Induktions-Schritt: Wir wollen die Aussage für $k+1$ zeigen, d.h. "Der Algorithmus gibt das Maximum der Liste korrekt zurück für Länge $k+1$ "

Da $k+1 > 1$ ist, wird der Algorithmus rekursiv mit Länge k ausgeführt

Die I.H. garantiert, dass $a' \in \{a_1, \dots, a_k\}$ und $a' \geq a_i$ für $i=1, \dots, k$

Case $a' > a_{k+1}$: Der Algorithmus gibt a' zurück

Bedingung (1) ist erfüllt, da $a' \in \{a_1, \dots, a_k\}$ per I.H

und somit gilt auch $a' \in \{a_1, \dots, a_{k+1}\}$

Bedingung (2) ist erfüllt, da $a' \geq a_i$ für $i=1, \dots, k$ per I.H

und somit gilt zusammen mit $a' > a_{k+1}$ auch $a' \geq a_i$ für $i=1, \dots, k+1$

Case $a' \leq a_{k+1}$: Der Algorithmus gibt a_{k+1} zurück

Bedingung (1) ist erfüllt, da $a_{k+1} \in \{a_1, \dots, a_{k+1}\}$

Bedingung (2) ist erfüllt, da $a' \geq a_i$ für $i=1, \dots, k$ per I.H

Zusammen mit $a_{k+1} \geq a'$ gilt: $a_{k+1} \geq a' \geq a_i$ für $i=1, \dots, k$

und somit $a_{k+1} \geq a_i$ für $i=1, \dots, k+1$

Somit ist die Aussage für alle $n \geq 1$ bewiesen per Induktion.

Exercise 2.3 *O*-notation quiz (2 bonus points). → Peergrading für heute

(a) For all the following functions the variable n ranges over $\mathbb{N}$. Prove or disprove the following statements. Justify your answer using Theorems 1 and/or 2.

(1) $2n^5 + 10n^2 \leq O(\frac{1}{100}n^6)$

(2) $n^{10} + 2n^2 + 7 \leq O(100n^9)$

(3) $e^{1.2n} \leq O(e^n)$

(4)* $n^{\frac{2n+3}{n+1}} \leq O(n^2)$ (This question does not count toward the bonus points.)

$$\lim_{n \rightarrow \infty} \frac{n^{\frac{2n+3}{n+1}}}{n^2} =$$

⇒ True

Exercise 2.5 Asymptotic growth of $\ln(n!)$.

Recall that the factorial of a positive integer n is defined as $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$. For the following functions n ranges over $\mathbb{N}_{\geq 2}$.

(a) Show that $\ln(n!) \leq O(n \ln n)$.

Hint: You can use the fact that $n! \leq n^n$ for $n \in \mathbb{N}_{\geq 2}$ without proof.

(b) Show that $n \ln n \leq O(\ln(n!))$.

Hint: You can use the fact that $\left(\frac{n}{2}\right)^{\frac{n}{2}} \leq n!$ for $n \in \mathbb{N}_{\geq 2}$ without proof.

$$\lim_{n \rightarrow \infty} \frac{n \ln n}{\ln(n!)}$$

Maximum Subarray Sum

Gegeben: Array der Länge n . Schreibweise $A[1, \dots, n]$ oder $(a_1, a_2, \dots, a_n)$

Gesucht: Startindex i und Endindex j , wobei $1 \leq i \leq j \leq n$,

sodass $\sum_{k=i}^j a_k$ maximal ist. (Die leere Summe ist zusätzlich auch erlaubt)

Beispiel: $[2, -5, 4, 1, -2, 7, -3, -14, 1, 2]$

$$\sum_{k=3}^6 a_k = 4 + 1 - 2 + 7 = \underline{\underline{10}}$$

Teilproblem: Randmaximum $R_i :=$ grösste Summe eines Subarrays, das in Index i endet

Beispiel: Array: $[2, -5, 4, 1, -2, 7, -3, -14, 1, 2]$

Randmaximum: $[2, -3, 4, 5, 3, 10, -7, 1, 3]$
 * * * (Subarray neu beginnen)

Lösung: $\max_{1 \leq i \leq n} R_i$ oder 0 falls alle Arrayelemente negativ sind

MSS-INDUKTIV($a_1, \dots, a_n$) Laufzeit: $\Theta(n)$

```

1 randmax  $\leftarrow$  0
2 maxS  $\leftarrow$  0
3 Für  $i \leftarrow 1, \dots, n$ :
4   randmax  $\leftarrow$  randmax +  $a_i$ 
5   Wenn randmax > maxS:
6     maxS  $\leftarrow$  randmax
7   Wenn randmax < 0:
8     randmax  $\leftarrow$  0
9 Gib maxS zurück.
```

- langsamere Algorithmen für MSS sind weniger wichtig. Sie wurden vor allem gezeigt, um euch die algorithmische Denkweise näher zu bringen

Θ und Ω - Notation

Intuitiv

$O(f)$:= Menge aller Funktionen, die asymptotisch langsamer oder gleich schnell wachsen

$\Theta(f)$:= Menge aller Funktionen, die asymptotisch gleich schnell wachsen

$\Omega(f)$:= Menge aller Funktionen, die asymptotisch gleich schnell oder schneller wachsen

Mathematisch

$$O(f) := \{ g: \mathbb{N} \rightarrow \mathbb{R}^+ \mid \exists C > 0 \forall n \in \mathbb{N} \ g(n) \leq C \cdot f(n) \}$$

$$\Theta(f) := \{ g: \mathbb{N} \rightarrow \mathbb{R}^+ \mid f \leq O(g) \text{ und } g \leq O(f) \} = O(f) \cap \Omega(f)$$

$$\Omega(f) := \{ g: \mathbb{N} \rightarrow \mathbb{R}^+ \mid f \leq O(g) \}$$

We can restate Theorem 1 from exercise sheet 2 as follows.

Theorem 1. Let N be an infinite subset of $\mathbb{N}$ and $f: N \rightarrow \mathbb{R}^+$ and $g: N \rightarrow \mathbb{R}^+$.

- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, then $f \leq O(g)$, but $f \neq \Theta(g)$. g wächst schneller (oder gleich schnell)
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = C \in \mathbb{R}^+$, then $f = \Theta(g)$.
and $g = \Theta(f)$ beide wachsen gleich schnell
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, then $f \geq \Omega(g)$, but $f \neq \Theta(g)$. f wächst schneller (oder gleich schnell)

• $g = \Theta(f)$ kann man alternativ auch zeigen, indem man $f \leq O(g)$ und $g \leq O(f)$

oder $f \leq O(g)$ und $f \geq \Omega(g)$

oder $g \leq O(f)$ und $g \geq \Omega(f)$

oder $f \geq \Omega(g)$ und $g \geq \Omega(f)$ zeigt.

Aufgaben:

True or False: $n + \log n = \Theta(n)$

obere Schranke: $n + \log n \leq$

untere Schranke: $n + \log n \geq$

daraus folgt: $n + \log n =$

anderer Lösungsweg: $\lim_{n \rightarrow \infty} \frac{n + \log n}{n} =$

True or False: $\log(n^5 + 3n) = \Theta(\log n)$

obere Schranke: $\log(n^5 + 3n) \leq$

untere Schranke: $\log(n^5 + 3n) \geq$

daraus folgt:

True or False: $2^n \geq \Omega(2^{n+3})$

True or False: $\sqrt{n^2 + 7n + 1} = \Theta(n)$

obere Schranke: $\sqrt{n^2 + 7n + 1} \leq$

untere Schranke: $\sqrt{n^2 + 7n + 1} \geq$

daraus folgt:

True or False: $\sin(n^3) + n \geq \Omega(n^2)$

$$\lim_{n \rightarrow \infty} \frac{\sin(n^3) + n}{n^2}$$

Counting function calls (your new favourite hobby)

Algorithm 1

$i \leftarrow 0$

while $i \leq 3n$ **do**

$f()$

$i \leftarrow i + 1$

$j \leftarrow 1$

while $j \leq n + 5$ **do**

$g()$

$g()$

$j \leftarrow j + 1$

Number of calls =

Algorithm 2

$i \leftarrow 1$

while $i \leq n$ **do**

$j \leftarrow 1$

while $j \leq n$ **do**

$f()$

$j \leftarrow j + 1$

$i \leftarrow i + 1$

$k \leftarrow 0$

while $k \leq 2n$ **do**

$g()$

$k \leftarrow k + 1$

Number of calls =

assume $n = k^2$ for some $k \in \mathbb{N}$

Algorithm 3 (more difficult)

$i \leftarrow 1$

Number of calls =

while $i \leq n^2$ **do**

$t \leftarrow 1$

while $t \leq i$ **do**

$f()$

$t \leftarrow t + 1$

$i \leftarrow i + 1$

$j \leftarrow 0$

while $j^2 \leq n$ **do**

$g()$

$j \leftarrow j + 1$

Beweismethode für Summen

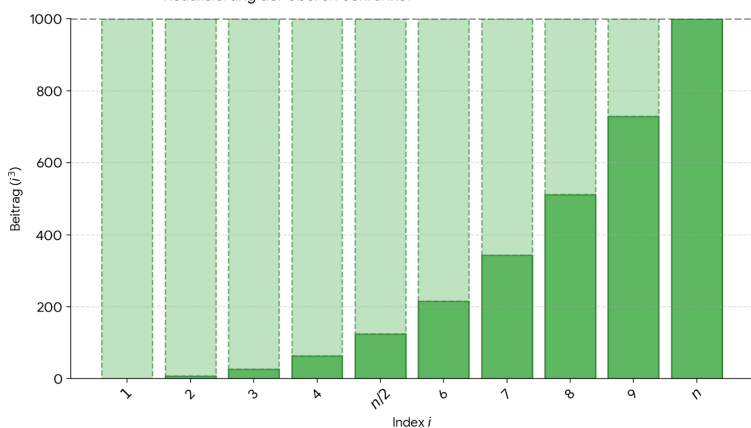
Show that $\sum_{i=1}^n i^3 = \Theta(n^4)$

obere Schranke: $\sum_{i=1}^n i^3 \leq$

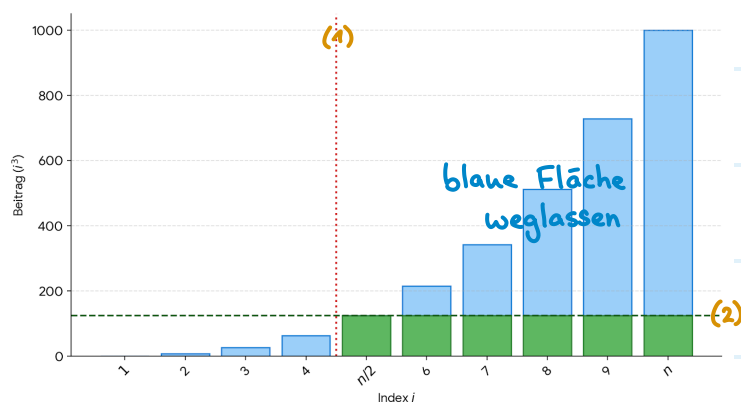
untere Schranke: $\sum_{i=1}^n i^3 \stackrel{(1)}{\geq}$

daraus folgt: $\sum_{i=1}^n i^3 = \Theta(n^4)$

Visualisierung der oberen Schranke:



Visualisierung der unteren Schranke:



Show that $\sum_{i=1}^n \sqrt{i} = \Theta(n^{1.5})$

obere Schranke: $\sum_{i=1}^n \sqrt{i} \leq$

untere Schranke: $\sum_{i=1}^n \sqrt{i} \geq$

daraus folgt: $\sum_{i=1}^n \sqrt{i} = \Theta(n^{1.5})$